

Real-Time Enveloped Mesh Editing Using Vector Graph Representation

Prashant Domadiya

Dhirubhai Ambani Institute of Information and
Communication Technology
India
pmdomadiya@gmail.com

Pratik Shah

Indian Institute of Information Technology Vadodara
India
pratik@iiitvadodara.ac.in

Abstract

In mesh processing, enveloping establishes a relationship between mesh and control structure deformation. Once enveloped, the mesh is edited via control structure deformation. This editing process aims for realistic and computationally efficient mesh deformation. In this work, we propose a data-driven enveloping approach designed for efficient mesh editing and producing realistic deformations. The proposed method computes geometric model deformations by leveraging the Vector Graph (VG) representation. By establishing a map between skeleton (control structure) and mesh deformations, the proposed approach achieves realistic results. Comparative analysis demonstrates qualitative parity with leading techniques, while computational efficiency remains akin to Linear Blend Skinning. Extensive experiments across diverse mesh datasets validate the proposed method, substantiated by qualitative and quantitative comparisons against state-of-the-art linear and non-linear approaches.

CCS Concepts

• **Computing methodologies** → **Animation; Mesh geometry models.**

Keywords

Enveloping, Geometric Model, Vector Graph Representation

ACM Reference Format:

Prashant Domadiya and Pratik Shah. 2024. Real-Time Enveloped Mesh Editing Using Vector Graph Representation. In *SIGGRAPH Asia 2024 Technical Communications (SA Technical Communications '24)*, December 03–06, 2024, Tokyo, Japan. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3681758.3698001>

1 Introduction

In graphics applications, mesh deformation is managed by manipulating its control structure through a process called enveloping. This process establishes a relationship between the mesh and its control structure using a mapping function, which can be linear or non-linear. Linear Blend Skinning (LBS) [Magenat-Thalmann

et al. 1989] is a widely used method due to its simplicity, real-time performance, and compatibility with high-performance hardware. In LBS, each vertex deforms independently based on the skeleton's deformation, but this may not preserve essential properties like edge length and face area, potentially leading to artifacts.

Various non-linear methods have been proposed for high-quality mesh deformations, incorporating multiple constraints to maintain inter-vertex relationships, such as edge length, for smooth deformations. However, these constraints make editing computationally expensive. Real-time performance is achieved by deforming vertex groups, known as patches, identified using clustering techniques like K-means [Jacobson et al. 2012] and mean-shift [James and Twigg 2005]. The effectiveness of this approach depends on the quality of segmentation. Ultimately, an enveloping method should balance computational efficiency and precise deformation to produce high-quality results.

In this work, we propose an enveloping method that achieves the quality of non-linear methods like [Jacobson et al. 2012; Wang et al. 2007; Xu et al. 2020] while maintaining the computational efficiency of LBS [Magenat-Thalmann et al. 1989]. Our method consists of two main steps: assigning a skeleton as a control structure to the mesh and establishing a map between skeleton and mesh deformations. With rigid bones, skeleton deformation is modeled as bone rotations. For mesh deformation, we utilize a Vector Graph (VG) representation [Domadiya et al. 2019], where individual vector deformation is modeled as rotation. Using deformed poses of the mesh from the dataset, we establish a deformation map that preserves inter-vertex relationships to the utmost degree without explicit constraints. Our method generates smooth, contextually relevant mesh deformations and allows real-time editing of high-resolution meshes due to the simplified relationship between skeleton and mesh deformations.

2 Vector Graph

Consider a triangular face of a mesh with vertices $[v_1, v_2, v_3]$, the vector tuple corresponding to the face is defined as, $[\vec{v}_1, \vec{v}_2, \vec{v}_3] = [v_1 - c_f, v_2 - c_f, v_3 - c_f]$, where $c_f = \frac{1}{3} \sum_{i=1}^3 v_i$. The vector graph of a mesh is a collection of vector tuples. The relationship between the mesh vertices and the vectors of the vector graph is defined as $VA = \vec{V}$. $V_{3 \times q}$ and $\vec{V}_{3 \times n}$ consist of q vertices and n vectors respectively, and the matrix $A_{q \times n}$ captures the mesh connectivity. For details, refer to [Domadiya et al. 2019]. Reconstructing a mesh from a given VG, involves solving $VA = \vec{V}$, an over-determined system of linear equations. It is solved by introducing a positional

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SA Technical Communications '24, December 03–06, 2024, Tokyo, Japan

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1140-4/24/12

<https://doi.org/10.1145/3681758.3698001>

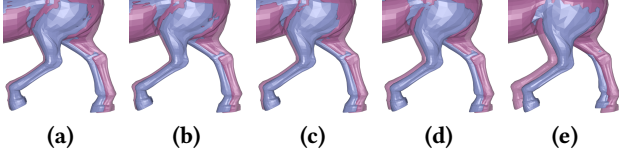


Figure 1: The impact of various α_2 and rotational parameters H_{ji} on the proposed enveloping method. The deformed mesh (in blue) is compared against the ground truth mesh (in pink). The deformed meshes (V_{df} defined in equation (9)) are generated by setting threshold $T = 0.1$, (a) $\alpha_2 = 0$, (b) $\alpha_2 = 0.25$, (c) $\alpha_2 = 0.5$, (d) $\alpha_2 = 0.75$ and (e) $\alpha_2 = 1.0$.

constraint,

$$\begin{aligned} \underset{V}{\operatorname{argmin}} \quad & \|VA - \bar{V}\|_2^2 \\ \text{subject to} \quad & V_C = y \end{aligned} \quad (1)$$

$$VA = [V_F, V_C] \begin{bmatrix} A_F \\ A_C \end{bmatrix} = V_F A_F + V_C A_C = \bar{V}, \quad (2)$$

$$V_F = (\bar{V} - V_C A_C) A_F^+ = \bar{V} A_F^+ - V_C A_C^+ \quad (3)$$

where $V_C = y \in \mathbb{R}^{3 \times |C|}$ and $V_F \in \mathbb{R}^{3 \times |F|}$ contain the constrained and the free vertices respectively, the matrix A_C contains rows of A corresponding to the constrained vertices whereas the matrix A_F contains the row of A corresponding to the free vertices. Note that The positions of constrained vertices (V_C) are known and V_C should contains at least one constrained vertex. The matrices $A_F^+ = A_F'(A_F A_F')^{-1}$ and $A_C^+ = A_C A_C'$ depend on the mesh connectivity and can be precomputed. The constrained vertices are appended later to V_F to get the estimated vertices ($V = [V_F, V_C]$) of the reconstructed mesh.

For the given reference and deformed meshes, we denote i^{th} reference and deformed vectors of a face by $\bar{v}_{ri}$ and $\bar{v}_{di}$ respectively. The transformation between the reference vector and deformed vector is defined as, $\bar{v}_{di} = \alpha_i J_i \bar{v}_{ri}$. The transformation (J_i) and the scaling (α_i) are computed as, $J_i = \mathcal{F}_{di} \mathcal{F}_{ri}'$ and $\alpha_i = \|\bar{v}_{di}\| / \|\bar{v}_{ri}\|$ where, $\mathcal{F}_{ri}$ and $\mathcal{F}_{di}$ are orthonormal frames defined as $\mathcal{F}_{ri} = [\bar{v}_{ri}, \bar{b}_{ri}, \bar{N}_{ri}]$, and $\mathcal{F}_{di} = [\bar{v}_{di}, \bar{b}_{di}, \bar{N}_{di}]$. Since these frames involved are orthonormal, the transformation matrix $J_i \in SO(3)$. The deformation between a reference mesh VG and deformed mesh VG is defined as,

$$\bar{V}_d = [\bar{v}_{d1}, \bar{v}_{d2}, \dots, \bar{v}_{dn}] = [\alpha_1 J_1 \bar{v}_{r1}, \alpha_2 J_2 \bar{v}_{r2}, \dots, \alpha_n J_n \bar{v}_{rn}]. \quad (4)$$

Finally, the deformed mesh V_d can be reconstructed from its VG representation $\bar{V}_d$.

3 Skeleton Assignment and Deformation

Initially, the user specifies the number of bones and joints for the skeleton (Figure 2(a)). They then select four vertices from the reference mesh for each joint. In Figure 2(b), selected vertices for joints x' and y' are depicted as crosses. Joint positions are updated by averaging the positions of these vertices. The Blender add-on fits the skeleton to the reference mesh by shifting the joints to their updated positions (Figure 2(c)) and assigns an orthonormal frame to each bone joint. The frame for the j^{th} bone is defined as $H_j = [\hat{u}_j, \hat{c}_j, \hat{e}_j]$, where $\hat{u}_j$ is parallel to the j^{th} bone, $\hat{c}_j = (e_j \times \hat{u}_j) / |e_j \times \hat{u}_j|$, and

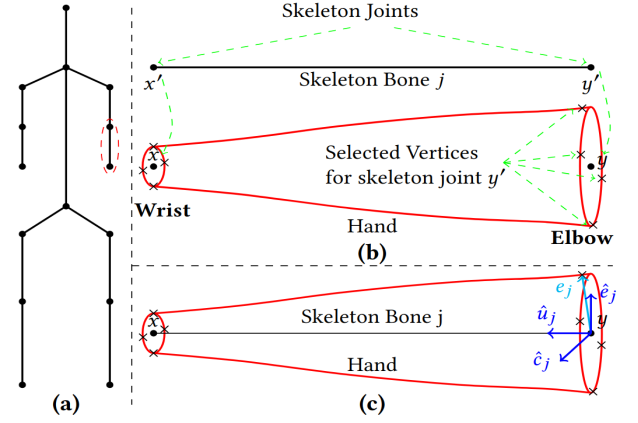


Figure 2: (a) User defined skeleton. (b) Skeleton bone assignment process. (c) Skeleton bone with orthonormal frame.

$\hat{e}_j = \hat{u}_j \times \hat{c}_j$. The vector e_j originates from joint j and extends toward one of the selected mesh vertices. The orthonormal frame is established at one joint per bone, with frames assigned to all bones. The relationship between mesh vertices and skeleton joints is $V_s = VU$. The skeleton is assigned to deformed poses of the reference mesh based on this relationship and extended to other deformed meshes. A step-by-step demonstration is provided in a supplementary video.

The transformation R_j between the j^{th} reference bone and deformed bone is defined as $R_j = H_{dj} H_{rj}'$ where, H_{rj} and H_{dj} are the reference and the corresponding deformed frames assigned to j^{th} bone. These frames are formed as, $H_{rj} = [\hat{u}_{rj}, \hat{c}_{rj}, \hat{e}_{rj}]$ and $H_{dj} = [\hat{u}_{dj}, \hat{c}_{dj}, \hat{e}_{dj}]$. Since both the frames are orthonormal and their determinants are +1, the transformation R_j is a rotation matrix. We have considered the rigid skeleton as the control structure. The relationship between the skeleton deformation and the mesh deformation is defined next.

4 Enveloping

In the proposed enveloping process, we define a map to characterize the deformation of the i^{th} vector in terms of the deformation of skeleton bones as,

$$\sum_{j=1}^b \omega_{ji} R_j^k H_{ji} = J_i^k, \quad k \in \{1, 2, \dots, p\} \quad (5)$$

where J_i^k is the rotation of the vector i as defined in section (2) and R_j^k is the rotation of the bone j as defined in section (3) corresponding to the k^{th} pose. $H_{ji} \in SO(3)$ is the rotational component of the map and $\omega_{1i}, \omega_{2i}, \dots, \omega_{bi}$ are non-negative affine weights associated with each bone of the skeleton. Since H_{ji} is the same for all available p poses, it can be estimated by solving,

$$\begin{aligned} \underset{H_{ji}}{\operatorname{argmin}} \quad & \alpha_1 \sum_{k=1}^p \|R_j^k H_{ji} - J_i^k\|_{\mathbb{F}}^2 + \alpha_2 \|H_{ji} - I_3\|_{\mathbb{F}}^2 \\ \text{subject to} \quad & H_{ji}' H_{ji} = I_3, \det(H_{ji}) = +1 \end{aligned} \quad (6)$$

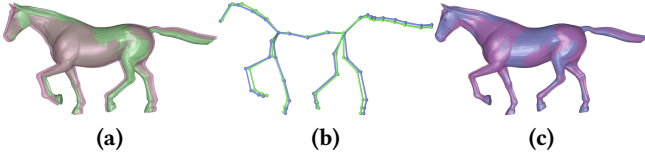


Figure 3: (a) Reference mesh (pink) and the deformed mesh (green) generated by skeleton bone rotation. (b) User-defined target skeleton (blue) and skeleton (green) derived from the deformed mesh. (c) The reference mesh (pink) and the mesh (blue) generated by applying both bone rotation and joint translations.

where, $\|\cdot\|_F$ is the Frobenius norm and I_3 is the identity matrix. α_1 and $\alpha_2 = 1 - \alpha_1$ are user defined convex weights. We solve equation (6) using the method suggested in [Eggert et al. 1997]. Next, the affine weights $\omega_{1i}, \omega_{2i}, \dots, \omega_{bi}$ are estimated by solving below optimization equations.

$$\begin{aligned} \underset{w_1, \dots, w_b}{\operatorname{argmin}} \quad & \sum_{k=1}^p \left\| \sum_{j=1}^b \omega_{ji} R_j^k H_{ji} \bar{v}_{ri} - J_i^k \bar{v}_{ri} \right\|_F^2 \\ \text{subject to} \quad & \omega_{ki} \in [T, 1], \sum_{k=1}^p \omega_k = 1 \end{aligned} \quad (7)$$

where $T \in [0, 1]$ is the user define threshold to constraint the weights. We solve equation (7) using the method from [Stark and Parker 1995]. The purpose of H_{ji} is to capture rotational differences between skeleton bone and vector rotation. As α_2 increases, H_{ji} approaches identity due to constraints, with a strict threshold T limiting bone allocation. To explore α_2 's impact, we vary it from 0 to 1, shown in Figure 1. At $\alpha_2 = 0$, H_{ji} accurately represents deformation, aligning the mesh with the ground truth. However, at $\alpha_2 = 1$, H_{ji} is identity, but the mesh misaligns. Optimal results are found with α_2 near 1, balancing alignment and identity transformation.

4.1 Simplification

Once the parameters of the map ψ_i , $i \in \{1, 2, \dots, n\}$ are estimated, the enveloped mesh can be deformed by editing the associated skeleton. The deformation of vector i is defined in (5) as $\bar{v}_{di} = J_i \bar{v}_{ri} = \sum_{j=1}^b \omega_{ji} R_j H_{ji} \bar{v}_{ri}$. Similarly, the other deformed vectors, v_{di} , are determined from the skeleton bone rotations. The deformed

Table 1: Quantitative comparison of various methods in terms of enveloping error and computation time. *ms* indicate time duration in milliseconds. μ and *Edt* stand for enveloping error and editing respectively.

Model	Verts q	Train. Poses p	Bones m	LBS		SSD		ARAP		RR		DM		Proposed	
				μ	Edt (ms)	μ	Edt (ms)	μ	Edt (ms)	μ	Edt (ms)	μ	Edt (ms)	μ	Edt (ms)
50002	6890	480	33	12.1	7.69	5.2	10.23	11.9	314	9.2	22.23	12.8	332	9.4	9.2
Horse	8431	11	40	27.1	9.07	14.4	21.27	29.8	430	30.8	29.30	36.7	412	20.3	11.50
Man	10002	175	20	30.8	5.21	13.2	11.48	31.1	533	25.7	16.91	31.0	23.8	23.8	7.54
Camel	21887	11	29	26.2	8.15	22.5	38.57	23.8	1067	23.2	30.64	24.0	14.5	482	11.15
Flamingo	26394	11	31	18.6	9.64	16.9	21.79	22.4	1377	18.0	48.07	22.2	16.1	638	13.75
Face	29299	11	656	13.5	253	49.7	163	13.2	4647	202	13.2	17.2	1360	23.1	162.64
Elephant	42321	11	42	14.9	24.68	31.8	100	12.2	4944	12.2	85.94	10.5	1990	7.5	16.58
Kid	59727	16	35	22.5	15.06	18.6	47.88	22.1	3386	22.1	86.37	22.3	2769	21.2	18.79

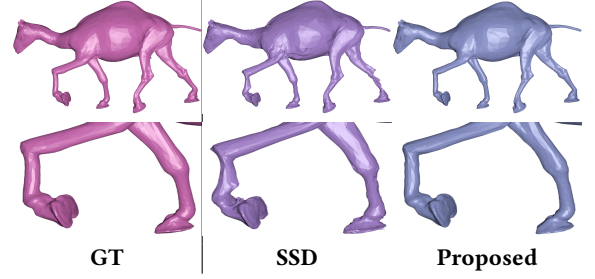


Figure 4: Comparison of the proposed method with SSD [Le and Deng 2012].

VG representation of the mesh is obtained as,

$$\bar{V}_d = [\bar{v}_{d1}, \bar{v}_{d2}, \dots, \bar{v}_{dn}] = [R_1, R_2, \dots, R_b]Q = R_d Q \quad (8)$$

where, Q is a sparse matrix since the deformation of the vector i depends on few skeleton bones only. The matrix R_d consists of rotations of deformed skeleton bones. The construction of Q is explained in supplementary material. The deformed mesh is reconstructed from its VG representation as described in section 2,

$$V_{dF} = \bar{V}_d A_F^+ - V_{dC} A_C^+ = R_d Q A_F^+ - V_{dC} A_C^+ = R_d \Phi. \quad (9)$$

V_{dC} is a constraint vertex which is set at origin 0_3 . The deformed mesh is obtained by combining deformed free and constrained vertices $V_d = [V_{dF}, V_{dC}] = [R_d \Phi, 0] = R_d M$ as described in section 2

4.2 Bone Translation

As shown in Figure 3(a), the deformed mesh (V_d in green) is generated via bone rotations, but it doesn't align perfectly with the ground truth mesh (in pink). This misalignment occurs because skeleton bone rotations alone can't achieve the desired mesh deformation. In Figure 3(b), the derived skeleton ($V_{sd} = V_d U$) doesn't match the user-defined skeleton (V_s). To correct this, skeleton joints should be shifted to their corresponding user-defined positions, and the mesh offset can be adjusted by translating the deformed mesh vertices accordingly. The adjusted deformed vertices are calculated as:

$$V_{dt} = V_d + X_s W_t = V_d + (V_s - V_{sd}) W_t \quad (10)$$

where $V_s - V_{sd}$ is the skeleton offset, and $W_t \in \mathbb{R}^{b \times q}$ contains non-negative convex weights. The vertex offset is the weighted sum of skeleton joint offsets. First, the reference mesh and skeleton are fitted within a unit cube, and geodesic distances from each vertex to all skeleton joints are computed. Joints are selected based on a user-defined distance threshold, with weights being the inverse of the geodesic distance, normalized by the sum of all inverses. This method ensures the refined deformed mesh (V_{dt}) closely fits the ground truth mesh, as demonstrated in Figure 3(c). By substituting $V_{sd} = V_d U$ and $V_d = R_d M$, equation (10) is further simplified to

$$V_{dt} = R_d M (I_n - U W_t) + V_s W_t = R_d \Gamma + V_s W_t \quad (11)$$

where, $\Gamma = M(I_n - U W_t) \in \mathbb{R}^{3b \times q}$ and $W_t \in \mathbb{R}^{b \times q}$ (computed once). I_q is an identity matrix of size q . The computational complexity of calculating V_{dt} is $O(3bq) + O(bq)$ which is similar to complexity LBS [Magnenat-Thalmann et al. 1989] ($O(4bq)$).

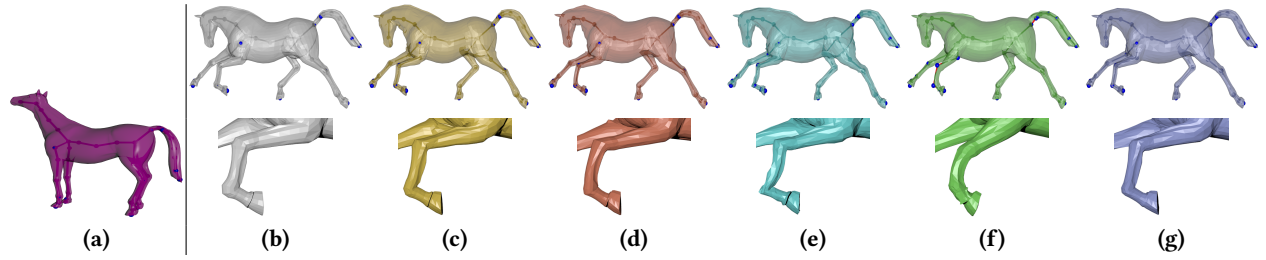


Figure 5: (a) Reference pose. (b) Ground truth pose from datasets. Estimated poses by (c) LBS [Magnenat-Thalmann et al. 1989] (d) ARAP [Jacobson et al. 2012] (e) RR [Wang et al. 2007] (f) DM [Mancewicz et al. 2014] and (g) proposed method.

5 Results and Experiments

In this section, we present the experiments and their results, both qualitatively and quantitatively. All experiments were conducted on an Intel i7 2.8GHz system with 8GB memory, using single-threaded Python implementations. In Table 1, the comparison is based on enveloping error (μ) and mesh editing time. Figure 5 presents qualitative evaluations of different enveloping methods. The assessment is done by comparing regenerated training meshes with the corresponding ground truth mesh in Figure 5. LBS [Magnenat-Thalmann et al. 1989] is the most computationally efficient, making it suitable for many applications, but it falls short in quality deformation, as shown in Table 1. In Figure 5(c), inconsistency artifacts are observed at the horses' forearm of meshes enveloped by LBS.

ARAP [Jacobson et al. 2012] addresses inconsistency artifacts by imposing an as-rigid-as-possible constraint in LBS [Magnenat-Thalmann et al. 1989] (Figure 5(d) and Table 1). The non-linear method RR [Wang et al. 2007] generates artifacts at the horse's leg (Figure 5(e)) but it ensures smooth deformation on other models (Table 1). DM [Mancewicz et al. 2014] smooths out LBS artifacts but shows curved bending artifacts (Figure 5(f)). These methods suffer from computational overload due to non-linearity as presented in Table 1. The proposed method is comparable in quality (Figure 5(g)) and ensures smooth, realistic deformation like nonlinear methods that to with computational efficiency similar to LBS.

SSD (Smooth Skinning Decomposition) [Le and Deng 2012] outperforms other methods on training meshes (Table 1) by executing real-time animations with minimal deformation error, but introduces joint artifacts in test poses (Figure 4). In Figure 6, the proposed method is compared qualitatively with Rignet [Xu et al. 2020] using a humanoid mesh after enveloping. The proposed method achieves more natural mesh deformation at the waist region tilt (top) compared to Rignet. Artifacts are noticeable at the right shoulder (top) and groins (bottom) of the humanoid mesh.

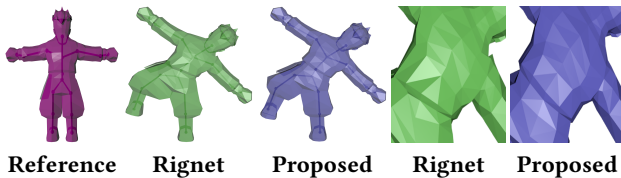


Figure 6: Comparison of the proposed method with Rignet [Xu et al. 2020].

6 Conclusion

This study demonstrates our data-driven enveloping method using the skeleton as a control structure, achieving smooth deformations without explicit constraints. The VG representation simplifies mesh editing, making it computationally similar to LBS. We compare our method with five state-of-the-art methods, showing it addresses inconsistency artifacts and is comparable to non-linear techniques while maintaining LBS-like efficiency. For interactive use, we developed a Blender 2.82 add-on that includes skeleton assignment, enveloping, and editing functionalities.

Acknowledgments

We express our deep gratitude to late Prof. Suman Mitra, whose invaluable contributions were instrumental in shaping this work.

References

- Prashant M Domadiya, Dr.Pratik Shah, and Suman Mitra. 2019. Guided Deformation Transfer. In *European Conference on Visual Media Production* (London, United Kingdom) (CVMP '19). Association for Computing Machinery, New York, NY, USA, Article 7, 10 pages. <https://doi.org/10.1145/3359998.3369408>
- D. W. Eggert, A. Lorusso, and R. B. Fisher. 1997. Estimating 3-D Rigid Body Transformations: A Comparison of Four Major Algorithms. *Mach. Vision Appl.* 9, 5-6 (March 1997), 272–290. <https://doi.org/10.1007/s001380050048>
- Alec Jacobson, Ilya Baran, Ladislav Kavan, Jovan Popović, and Olga Sorkine. 2012. Fast Automatic Skinning Transformations. *ACM Trans. Graph.* 31, 4, Article 77 (July 2012), 10 pages. <https://doi.org/10.1145/2185520.2185573>
- Doug L. James and Christopher D. Twigg. 2005. Skinning Mesh Animations. *ACM Trans. Graph.* 24, 3 (July 2005), 399–407. <https://doi.org/10.1145/1073204.1073206>
- Binh Huy Le and Zhigang Deng. 2012. Smooth Skinning Decomposition with Rigid Bones. *ACM Trans. Graph.* 31, 6, Article 199 (Nov. 2012), 10 pages. <https://doi.org/10.1145/2366145.2366218>
- N. Magnenat-Thalmann, R. Laperrière, and D. Thalmann. 1989. Joint-Dependent Local Deformations for Hand Animation and Object Grasping. In *Proceedings on Graphics Interface '88* (Edmonton, Alberta, Canada). Canadian Information Processing Society, CAN, 26–33.
- Joe Mancewicz, Matt L. Derksen, Hans Rijpkema, and Cyrus A. Wilson. 2014. Delta Mush: Smoothing Deformations While Preserving Detail. In *Proceedings of the Fourth Symposium on Digital Production* (Vancouver, British Columbia, Canada) (DigiPro '14). Association for Computing Machinery, New York, NY, USA, 7–11. <https://doi.org/10.1145/2633374.2633376>
- Philip B Stark and Robert L Parker. 1995. Bounded-variable least-squares: an algorithm and applications. *Computational Statistics* 10 (1995), 129–129.
- Robert Y. Wang, Kari Pulli, and Jovan Popović. 2007. Real-time Enveloping with Rotational Regression. In *ACM SIGGRAPH 2007 Papers* (San Diego, California) (SIGGRAPH '07). ACM, New York, NY, USA, Article 73. <https://doi.org/10.1145/1275808.1276468>
- Zhan Xu, Yang Zhou, Evangelos Kalogerakis, Chris Landreth, and Karan Singh. 2020. Rignet: Neural Rigging for Articulated Characters. *ACM Trans. Graph.* 39, 4, Article 58 (July 2020), 14 pages. <https://doi.org/10.1145/3386569.3392379>